

Soham Kotkar Task:

Sovereign Language Expansion — Lightweight MCP + RL pipeline

Goal: Add ~18 new low-resource languages into the Sovereign LM Bridge by streaming micro-corpora (no heavy local downloads), training tiny LoRA adapters on streaming data, and feeding RL episodes into the MCP pipeline. Deliver a working pipeline + smoke tests for each language.

Duration: 6 days (agile micro-sprints)

High-level plan (what to produce)

1. adapter_service/ — streaming LoRA trainer (no large local datasets)
2. mcp_connectors.yml — remote sources & scrapers for target languages
3. scrapers/ — small scraping helpers for low-resource content (news, wiki, gov PDFs)
4. rl/ — episode collector + upload helper (push episodes to NAS/S3)
5. bridge/tests/smoke_results.md — 10 prompts per language (sanity)
6. PR into branch task_sov_lang_expand with README + runbook

Day-by-day step-by-step (copy/paste ready)

Day 0 — Prep (1–2 hours)

- Create branch and empty commit

```
git checkout -b task_sov_lang_expand
git commit --allow-empty -m "start: sovereign language expansion"
git push -u origin task_sov_lang_expand
```

- Create folder skeleton:

```
mkdir -p adapter_service mcp scrapers rl bridge/tests
```

- Create mcp_connectors.yml with placeholders:

```
sources:
  - id: "hf_stream:open_dataset_langX"
    type: "huggingface_stream"
    uri: "dataset/ref:split"
  - id: "http_feed:wikimedia_lang"
    type: "http_pages"
    base_url: "https://<lang>.wikimedia.org/wiki/
Special:AllPages"
  - id: "s3_raw:gov_docs"
    type: "s3"
    bucket: "s3://gurukul-raw/lang/<lang>"
```

- Commit:

```
git add .
git commit -m "chore: scaffold for sovereign language
expansion"
git push
Contacts: Ask Yaseen/Karan for any scraping proxies or stable endpoints if blocked.
```

Day 1 — Scrapers + streaming connectors (6–8 hrs)

Goal: Produce small streaming scrapers and MCP stream client.

1. Add scrapers for 3 priority languages (pick closest/most data first: e.g., Hindi dialects, Bengali, Tamil).
 - File: scrapers/scrape_wiki.py (simple page fetch + text extraction)
 - File: scrapers/scrape_news.py (RSS → fetch → extract text)
2. Add a streaming client: mcp/stream_client.py:
 - Accepts mcp_connectors.yml, yields cleaned sentences (generator).
 - Use datasets.load_dataset(..., streaming=True) where available.
3. Implement small cleaning: Unicode normalization, minimal token-level filters, language tag prefix <lang:xx>.
4. Test streaming locally for one source:

```
python -c "from mcp.stream_client import stream;
print(next(stream('hf_stream:some')))"
```

5. Commit & push.

Acceptance Day1: mcp/stream_client.py yields sentences for at least 3 languages; run prints a sample sentence.

Day 2 — Adapter trainer (streaming LoRA) (6–8 hrs)

Goal: Train tiny LoRA adapters using streaming input; must run on 4050/3080 with 8-bit.

1. Create adapter_service/train_adapt.py — behavior:
 - Load base model (configurable) via --model_name_or_path.
 - Stream N samples from mcp (use --max_train_samples, default 2000).
 - Use bitsandbytes + peft LoRA with 8bit=True.
 - Use --gradient_accumulation_steps to fit VRAM.
 - Save adapter to adapters/<lang>_lite.
2. Example run command:

```
python adapter_service/train_adapt.py \
  --model_name_or_path "gpt-small-base" \
  --adapter_out adapters/awadhi_lite \
  --stream_source "mcp:wiki_awadhi" \
  --max_train_samples 1500 \
  --num_epochs 3 \
  --per_device_train_batch_size 1 \
  --gradient_accumulation_steps 8 \
  --use_8bit True
```

3. Add adapter_service/model_utils.py to load base model + apply adapter at inference.
4. Commit & push.

Acceptance Day2: One adapter trained for one low-resource language (e.g., Awadhi); saved under adapters/awadhi_lite.

Day 3 — FastAPI inference + generate-lite (4–6 hrs)

Goal: Lightweight REST endpoints to test adapters.

1. Add adapter_service/api.py with endpoints:
 - POST /adapter/train-lite — triggers train_adapt.py via subprocess (async job).

- POST /generate-lite — accepts {prompt, lang, max_tokens} and returns generated text (loads base model + adapter).
- GET /adapter/status/{job_id} — return job logs.

2. Run locally:

```
uvicorn adapter_service.api:app --host 0.0.0.0 --port 8100 --
reload
curl -X POST http://localhost:8100/generate-lite -d
'{"prompt":"राम का परिचय","lang":"hi"}' -H 'Content-Type:
application/json'
```

3. Add minimal rate-limiter (per-ip) config.

Acceptance Day3: generate-lite returns sensible output for 5 prompts across 3 languages (smoke).

Day 4 — RL episode collector + upload (4–6 hrs)

Goal: Collect episodes (prompt, output, auto-reward) and push to NAS/S3 for cloud trainer.

1. Implement rl/collect.py:

- Collect records: {trace_id, prompt, output, timestamp, auto_reward}.
- Auto-reward heuristics: language match, presence of KB citation token, simple perplexity delta or edit-distance metric.
- Save to rl/episodes/<lang>/episode_<ts>.jsonl.

2. Add rl/upload_helper.py to push episodes to S3/NAS:

```
python rl/upload_helper.py --local_path rl/episodes/ --
s3_path s3://bhiv/rl_episodes/soham/
```

3. Connect to Yotta / NAS:

- If Vijay unavailable: talk to Yaseen/Karan for S3 credentials.

4. Commit & push.

Acceptance Day4: 200 episodes generated and uploaded to the configured NAS/S3 path.

Day 5 — Language expansion loop + smoke tests (6–8 hrs)

Goal: Run end-to-end micro-pipeline for 8–10 target languages and produce smoke reports.

1. For each target language (start order: Awadhi, Bhojpuri, Magahi, Chhattisgarhi, Haryanvi, Himachali, Pahadi, Sinhala, Tamil-SriLanka, Tibetan, Dzongkha, Mizo, Pashto, Dari, Vietnamese, Thai, Burmese, Vietnamese):

- Trigger train-lite (small), run generate-lite with 10 test prompts (from bridge/tests/prompts_<lang>.json).
 - Collect outputs and run auto-eval metrics (language tag accuracy, simple BLEU vs tiny reference if exists).
2. Populate bridge/tests/smoke_results.md with table: language, success_rate, avg_latency, adapter_path, episodes_uploaded.
 3. Run the MCP streaming for each language for 30 min to collect data and run a second tiny adapter update from that stream.
 4. Commit & push.

Acceptance Day5: smoke_results.md shows at least 10 languages with sensible outputs and average latency <1s for generate-lite.

Day 6 — Integration, docs, PR & handover (4–6 hrs)

Goal: Finalize docs; create PR; hand off episodes + adapter artifacts to Task Bank.

1. Add README.md with runbook:
 - How to run training, generate, collect episodes, upload to S3.
 - How to test end-to-end (curl examples).
2. Push adapters to NAS adapters/ if size allows, or publish metadata & S3 path.
3. Create PR to task_sov_lang_expand with HDIG reflection:
 - Humility: one limitation
 - Gratitude: one help
 - Integrity: one risk/shortcut
4. Notify: Vinayak (Task Bank) with short message and attach the PR link & s3 paths.

Acceptance Day6: PR created, CI basic checks pass, bridge/tests/smoke_results.md present, episodes uploaded, adapters available on NAS/S3. Notify Vinayak for validation.

Scripts & example commands (quick copy-paste)

Train adapter (streaming small):

```
python adapter_service/train_adapt.py \
  --model_name_or_path "gpt-small-base" \
  --adapter_out adapters/bhojpuri_lite \
  --stream_source "mcp:wikipedia_bhojpuri" \
  --max_train_samples 1200 \
```

```
--num_epochs 3 \  
--per_device_train_batch_size 1 \  
--gradient_accumulation_steps 8 \  
--use_8bit True
```

Run API (local):

```
uvicorn adapter_service.api:app --host 0.0.0.0 --port 8100 --  
reload  
curl -s -X POST http://localhost:8100/generate-lite \  
-H "Content-Type: application/json" \  
-d '{"prompt":"कृषि पर संक्षेप में बताइए","lang":"bho"}'
```

Collect episodes and upload:

```
python rl/collect.py --source http://localhost:8100/generate-  
lite --prompts bridge/tests/prompts_bhojpuri.json  
python rl/upload_helper.py --local_path rl/episodes/ --  
s3_path s3://bhiv/rl_episodes/soham/
```

Create PR & notify (example):

```
git add .  
git commit -m "feat: streaming adapters + RL episodes for low  
resource langs"  
git push origin task_sov_lang_expand  
# Open PR and add HDIG reflection in description
```

Heavier dependencies & fallbacks

- If a language has no streaming source, use small web-scrape (news, wiki, government pages). Keep scraped corpus <100MB per language.
- If Yotta/S3 credentials delayed, upload episodes to Nipun's Qdrant bucket or a secure NAS path and notify Vinayak.
- If VRAM issues on 4050: reduce max_train_samples, increase gradient_accumulation_steps, reduce num_epochs.

Contacts & responsibilities (who to ping)

- Dataset / scraping access & proxies: Yaseen / Karan
- S3 / NAS for episode uploads: Yaseen / Karan (if Vijay unavailable) — otherwise Vijay
- Core & MCP connector questions: Akash / Nisarg (Nisarg to confirm Core mapping; Akash to validate endpoints)

- Vaani prosody mapping (tone fields): Karthikeya
- Task Bank validation & packaging: Vinayak Tiwari

Acceptance Criteria (final checklist)

- Branch `task_sov_lang_expand` created and PR opened.
- `mcp/stream_client.py` returns streamed sentences for target languages.
- At least 8 new adapters trained (tiny LoRA) and saved to adapters/ or uploaded to NAS.
- `adapter_service/api.py` running with `generate-lite` returning sensible outputs for 10 prompts $\times$ languages.
- `rl/episodes/` contains ≥ 200 episodes and uploaded to S3/NAS path.
- `bridge/tests/smoke_results.md` with per-language metrics (latency, pass/fail).
- README + runbook + HDIG reflection included in PR.
- Notify Vinayak & create Task Bank entry with links and brief test instructions.